



Pharma Pulse

A Java – Based Pharmacy Management System

VITyarthi Project Report

Student Name : Madhvesh Dhirenkumar Kapadia
Registration Number : 24BEC10078
Course / Subject : Programming in Java
Branch : Electronics & Communication Engineering
Repository : <https://github.com/Madhvesh-12/pharmapulse>

Table of Contents

Sr. no.	Content
1.	Introduction
2.	Problem statement
3.	Functional Requirements
4.	Non – Functional Requirements
5.	System Architecture
	5.1 Presentation Layer
	5.2 Service Layer
	5.3 Concurrency Layer
	5.4 Data Access Layer
	5.5 Persistence Tier
6.	Design Diagrams
	6.1 Use Case Diagram
	6.2 Workflow / Process Flow Diagram
	6.3 Sequence Diagram – Placing an Order
	6.4 Class Diagram
	6.5 ER Diagram
7.	Key Design Decisions & Implementations
	7.1 Abstract Class <i>Item</i>
	7.2 <i>Billable</i> Interface
	7.3 Custom <i>InsufficientStockException</i>
	7.4 Worker Thread Synchronizarion
	7.5 Lambda-Driven Database Execution
8.	Testing Approach and Verification
9.	Execution Screeshots
10.	Challenges & Engineering Takeaways
11.	Future Scope
12.	References

1. Introduction:

PharmaPulse is a consol-based pharmacy management program written in Core Java and uses MySQL as its database. The system was developed to mimic daily retail pharmacy business by maintaining inventory in store, accepting customers' prescription orders, avoiding over selling of items, and providing receipts to customers.

Unlike other assignments, where course contents were discussed individually, this assignment tries to incorporate object-oriented modelling, custom checked exceptions, multithreading, collections, file streams and JDBC.

2. Problem Statement:

Small-medium retail pharmacies often have issues regarding inventory reconciliation whenever there is rapid sales activity. Manual ledger or simple non-concurrent software can be responsible for race condition which is where sales of the same remaining medicine occur simultaneously thus causing mismatch and even negative stock balances.

PharmaPulse prevents this problem through the synchronization of checkout process and database operation by ensuring that stock validation occurs before payments and logging and stock deduction occur concurrently in a background worker thread while receipts are exported automatically to the local drive.

3. Functional Requirements:

- **Inventory Tracking:** Full CRUD support to add new medicine batches, browse current stock levels, update pricing or quantities and deleting obsolete records.
- **Order Processing & Cart Management:** A shopping cart workflow where cashiers can add multiple products with variable quantities in one transaction.
- **Defensive Stock Check:** Order cannot be processed if requested quantities exceeds current stock by throwing a custom *InsufficientStockException* before touching the database.
- **Thread-Safe Order Persistence:** Asynchronously executes the order logging and decrements product inventory using a synchronized worker thread (*OrderProcessorThread*).
- **Billing & Invoice Generation:** Computes order totals and automatically writes a clean text receipt (*invoice_<CustomerName>.txt*) to disk using Java Buffered character streams.

4. Non – Functional Requirements:

- **Data Integrity & Thread Safety:** Critical processes (order insertion and stock decrement) have been synchronized using the worker thread class to prevent any dirty write or lost update in case of multiple transactions.
- **Defensive Resource Management:** All database queries use try-with resources to make sure that connections, prepared statements, and result sets close instantly to avoid any connection leakage.
- **Prevention of SQL Injection:** All queries are processed through parameterized *PreparedStatement* objects only.
- **Modularity in Code:** Proper segregation of layers among the presentation layer (*PharmaPulseApp*), model objects (*Product* and *Order*), service layer (*ProductService* and *OrderService*), and persistence layer (*DBConnection*).
- **Access Restriction:** Controlled by an administration passkey ('*pp1234*') before accessing the main menus.

5. System Architecture:

PharmaPulse has a tiered architecture that decouples the components responsible for dealing with user interaction, business logic, multithreading capabilities, and data access:

5.1 Presentation Layer (*PharmaPulseApp*):

Takes care of menu interactions, verifies the user input, checks the administrator passcode('pp1234') and forward the user decision to the service layer.

5.2 Service Layer (*ProductService* & *OrderService*):

Handles all the business logic, creates Product and Order Objects, checks current inventory levels and throws *InsufficientStockException* in case of shortage, generates invoice output format.

5.3 Concurrency Layer (*OrderProcessorThread*):

Places the order in the background by executing the order insertion operation and decrement in block stock level inside synchronized block of code to avoid race conditions.

5.4 Data Access Layer (*DBConnection*):

Reads credentials from *db.properties* file, closes connections automatically and delegates all JDBC statements to function dispatchers (*SQLConsumer*, *SQLFunction*) to avoid repetitive code.

5.5 Persistence Tier:

Persists inventory and sales in tabular form in MySQL database (*products* and *orders*), as well as writes customer invoices to disk in '.txt' format using *BufferedWriter*.

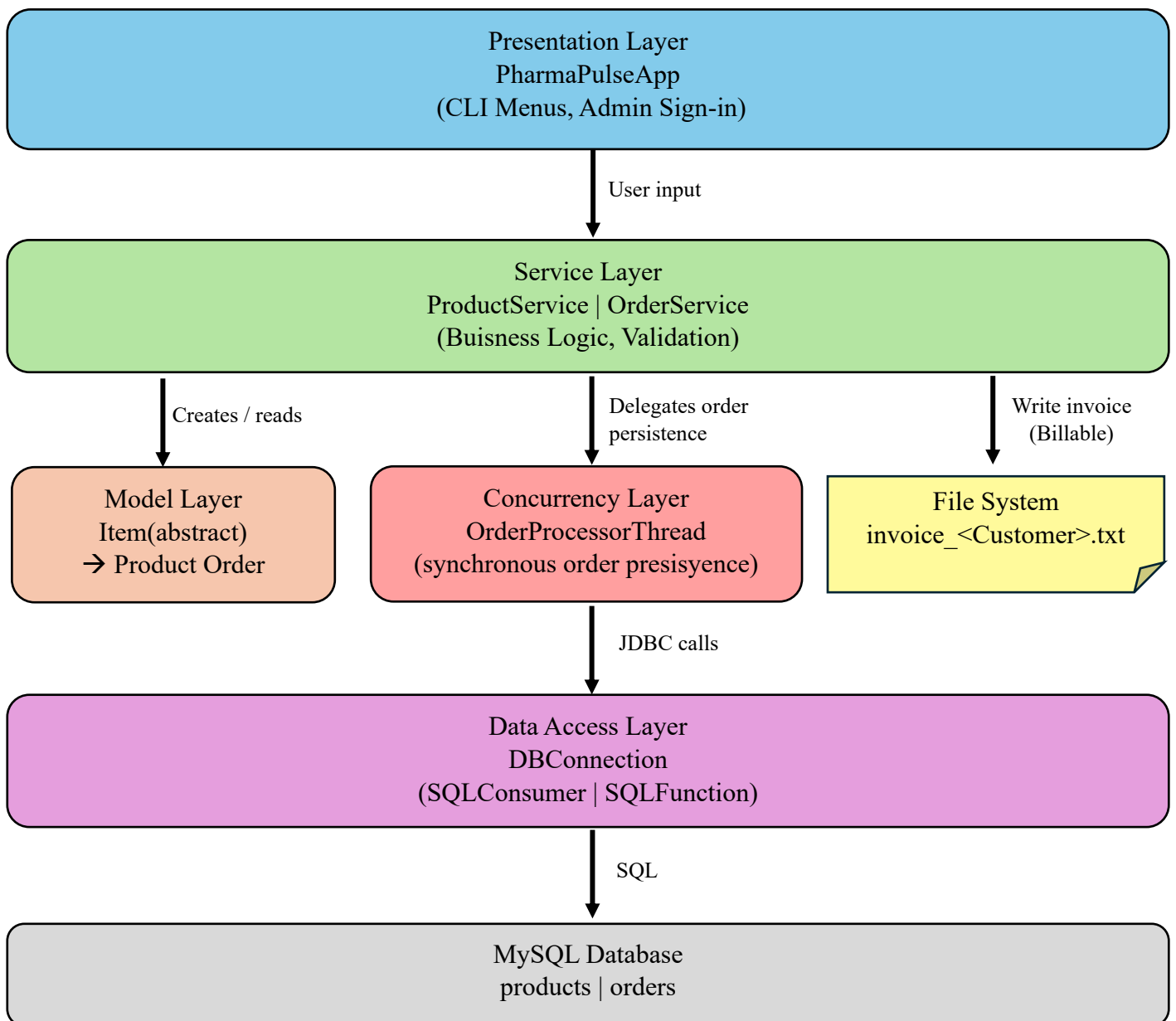


Fig. 1 – System Architecture

6. Design Diagrams:

6.1 Use Case Diagram:

The system has a single actor, the Admin/Pharmacist, and nine use cases inside the system boundary. They fall into three groups:

- **Access:** Sign In with Access Code
- **Inventory:** View, Add, Update and Remove Product
- **Orders & Billing:** View Orders, Create Order, Cancel Order and Generate Invoice

Signing in is a precondition for everything else, since the menus can't be reached without a valid access code. Create Order is the richest use case: it includes a stock check, and it is the only one with a business-rule failure path, which raises *InsufficientStockException*. The database and file system sit behind the boundary as internal components, not actors. I kept a single actor because role separation (admin vs. cashier) is listed under Future Enhancements.

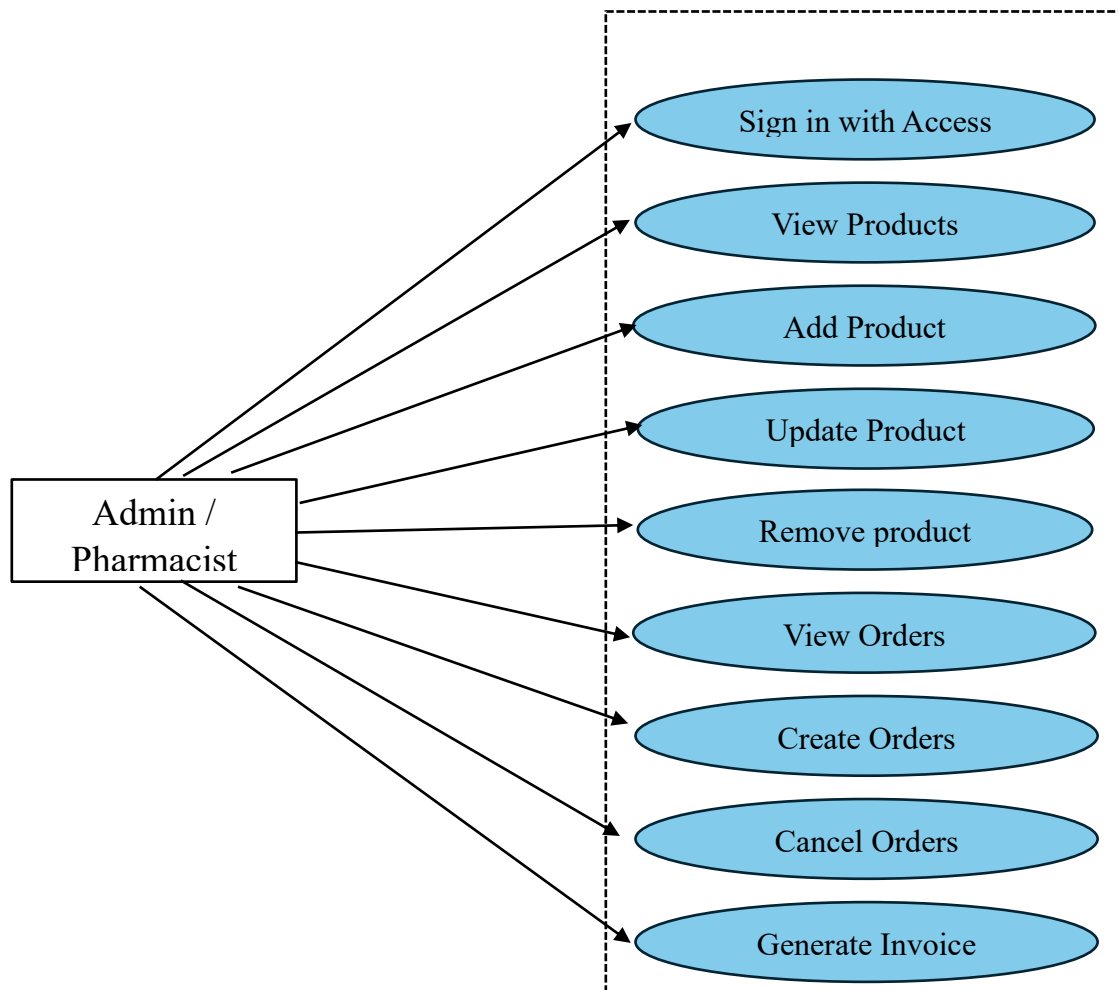


Fig.2 – Use Case Diagram

6.2 Workflow / Process Flow Diagram:

The workflow follows the program from launch to exit:

- The app starts and asks for the admin access code.
- If the code is invalid, the flow loops back to the prompt. If it is valid, the Main Menu opens.
- From the Main Menu the admin picks **1** (Inventory Menu), **2** (Orders & Billing Menu) or **0** (terminate).
- Each submenu has a **0 (return)** option that goes back to the Main Menu, so navigation is a loop and the session only ends when the admin chooses 0 at the top level.

Two branches deserve attention. Choosing **New Order** spawns an `OrderProcessorThread` that persists the order and updates stock, and choosing **Invoice** writes `invoice_<name>.txt` to disk. These are the only two places where the program leaves the plain "read input, call service, print result" pattern.

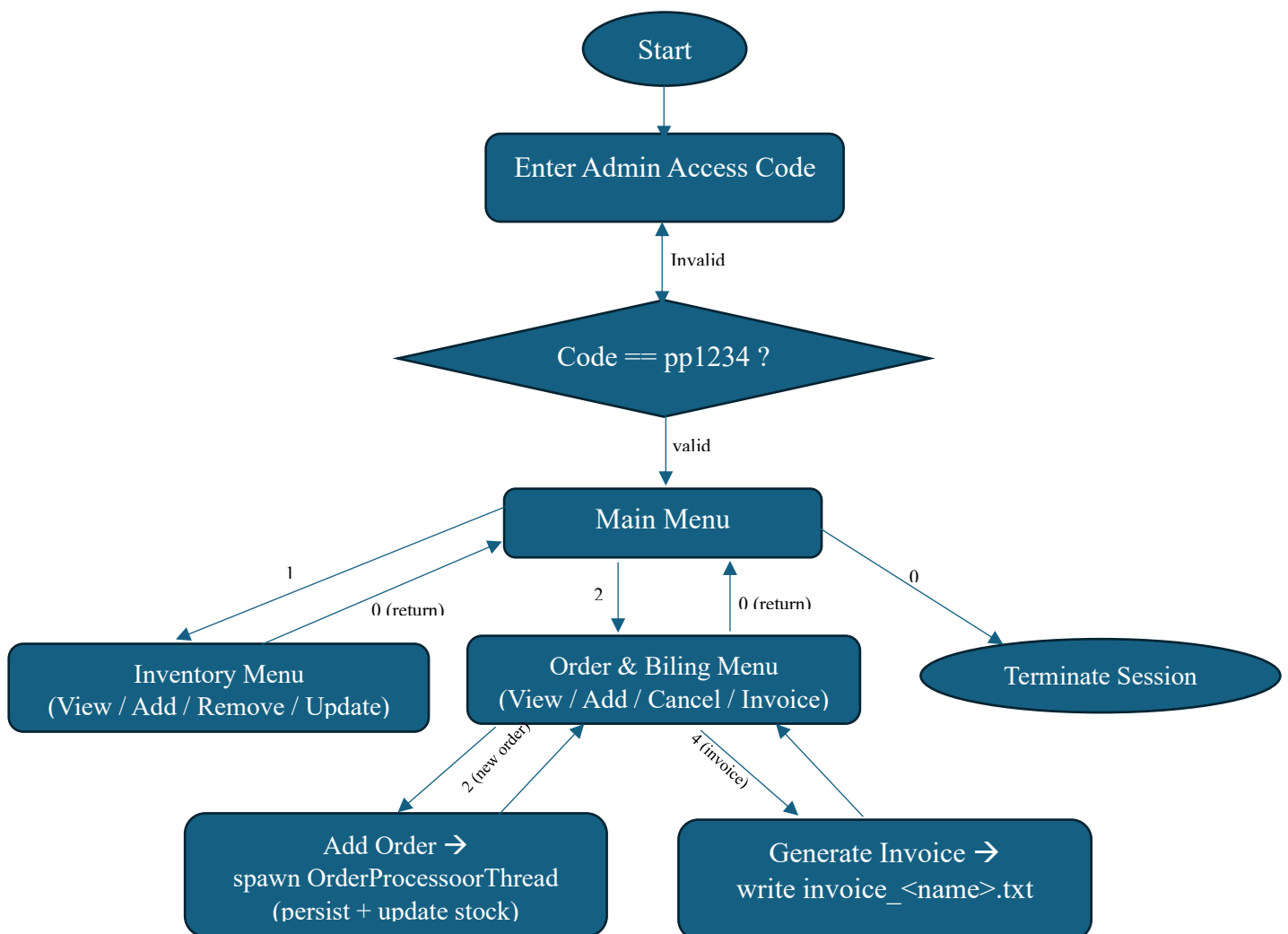


Fig.3 – Workflow Diagram

6.3 Sequence Diagram – Placing an order:

This diagram shows the order in which the objects interact when *addOrder()* runs:

1. The admin builds the cart (product names and quantities) through *OrderService*.
2. *OrderService* asks *DBConnection* for the current PName, Cost and Stock values, and MySQL returns the result set.
3. **Validation happens here, on the main thread.** If any requested quantity exceeds stock, an *InsufficientStockException* is thrown and nothing is written.
4. If stock is sufficient, *OrderService* creates and starts a new *OrderProcessorThread*.
5. Inside a synchronized block, the thread runs *persistOrder()*, which does an INSERT INTO orders and then UPDATE products SET Stock = Stock - ?.
6. The main thread calls *join()*, waits for the thread to finish, and only then logs SUCCESS with the order total.

Three design decisions are visible here. Validating before starting the thread avoids opening a database connection for orders that will be rejected. The synchronized block makes the insert and stock update one uninterrupted unit, which prevents race conditions. And *join()* guarantees the success message is only shown after the data is actually saved.

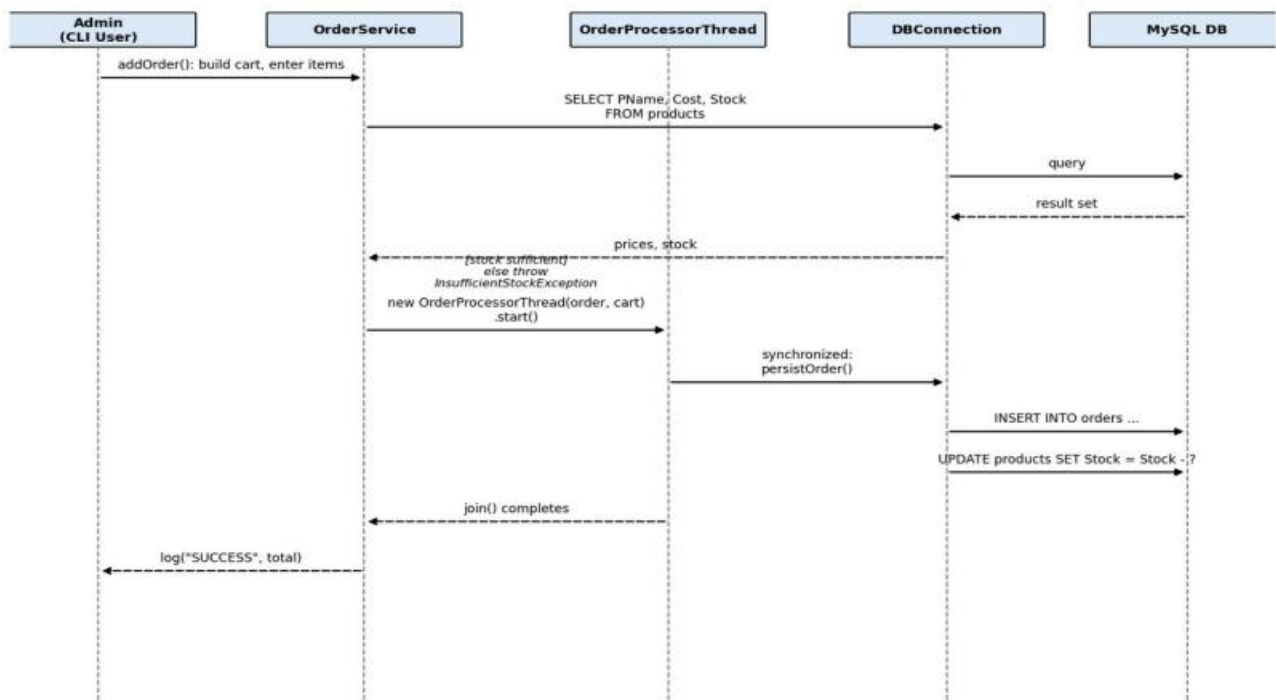


Fig.4 – Sequence Daigram (Create Order)

6.4 Class Diagram:

The class diagram maps directly onto the Java concepts from the course:

Relationship in the diagram	Concept it demonstrates
<i>Product</i> extends abstract <i>Item</i>	Inheritance and abstraction
<i>OrderService</i> implements <i>Billable</i>	Interfaces and polymorphism
<i>OrderProcessorThread</i> extends <i>Thread</i>	Multithreading
<i>InsufficientStockException</i> extends <i>Exception</i>	Custom checked exceptions
<i>OrderService</i> throws <i>InsufficientStockException</i>	Exception handling
Services depend on <i>DBConnection</i>	Separation of concerns and functional interfaces
Private fields with public getters (<i>Order</i> , <i>Product</i>)	Encapsulation

Item holds shared fields (id, name, cost); *Product* adds brand, productType, and stock. *Billable* separates billing behaviour (*calculateDiscount*, *exportInvoiceToFile*) from order management. *OrderService* creates the thread, and both services use *DBConnection* (*getConnection*, *execute*, *executeQuery*) instead of opening connections themselves.

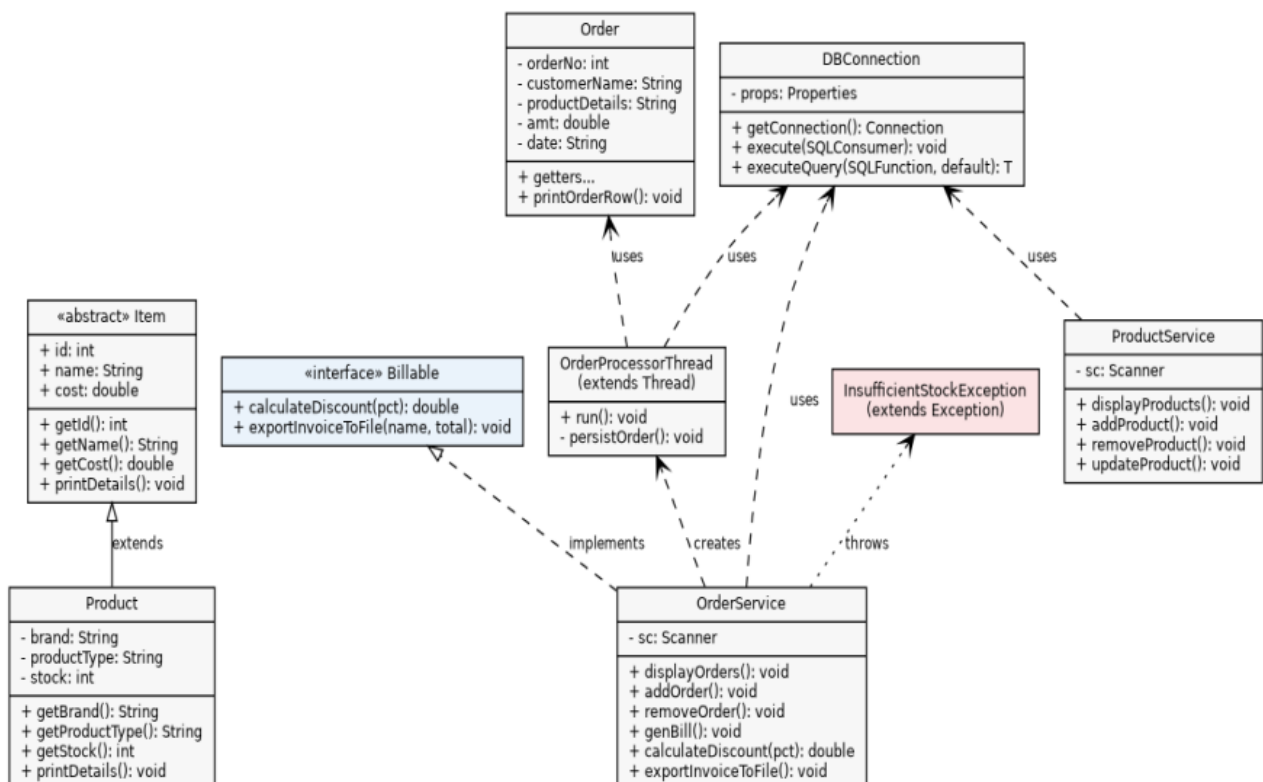


Fig 5 – Class Diagram

6.5 ER Diagram:

The database has two tables:

- **Products:** PID (primary key), PName, Brand, PType, Stock, Cost
- **Orders:** OrderNo (primary key), CName, the ordered items stored as a PName:Qty text field, Amt, Date

Logically, one order can contain many products and one product can appear in many orders, which is a many-to-many relationship. Because the items are stored as a formatted string, that relationship is only implied, shown by the dashed line, and can't be enforced with a foreign key. This is the main limitation of the current schema. The planned fix is an *OrderItems* table (OrderNo, PID, Quantity) that resolves the many-to-many relationship properly and gives the database real referential integrity.

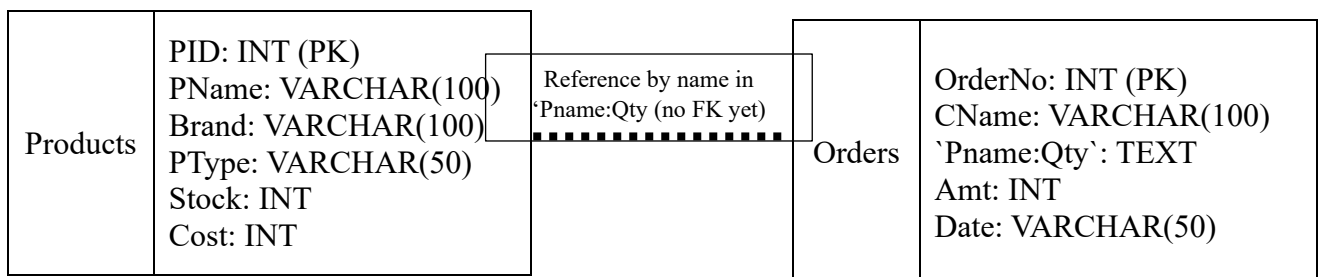


Fig. 6 – ER Diagram

7. Key Design Decisions & Implementations:

7.1 Abstract Class *Item*:

Establishes shared attributes (*id*, *name*, *cost*) and define *printDetails()*. *Product* extends this base class by adding batch attributes (*brand*, *productType*, *stock*).

7.2 *Billable* Interface:

Decouples the billing and receipt writing contract from standard product management. *OrderService* implements this to handle discount logic and the *exportInvoiceFile()* method.

7.3 Custom *InsufficientStockException*:

Implemented as a checked exception extending *java.lang.Exception*. Because attempting to purchase out-of-stock items is a standard real-world senerio, making it a checked exception forces the application to handle stock errors gracefully without crashing.

7.4 Worker Thread Synchronization:

Order execution is handled by *OrderProcessorThread* extending *Thread*. Inside this *run()* method, a *synchronized* (*OrderProcessorThread.class*) block ensures database updates and stock adjustments execute as an atomic sequence.

7.5 Lambda-Driven database Execution:

Instead of rewriting repetitive *try/catch* blocks across all queries, *DBConnection* implements functional interface (*SQLConsumer*, *SQLFunction*). This centralizes error handling and ensures connection pooling cleanups happen automatically.

8. Testing Approach & Verification:

Testing was conducted against a live local MySQL database across core user flows:

- **Inventory Checking:** Additional products (Paracetamol, Asthakind) were included, updated the inventory numbers, and ensured that the alignment in the table is still correct in the terminal.
- **Order Entry and Inventory Decrease:** Placed an order of 5 units of the medicine in which there are 50 units of the medicine in stock. It was checked that the order was placed and the stock decreased to 45 units instantly.
- **Exception Handling:** Tried to place an order for 100 units of the product but there were only 45 units available. *InsufficientStockException* was raised, caught and displayed a warning message.
- **Invoice Generation:** Invoke the bill generation functionality for an existing customer. It was checked that *invoice_<CustomerName>.txt* file was generated successfully.

9. Execution ScreenShots:

- **Admin login prompt with *pp1234* entered and successfully access granted**

```
C:\Users\madhv\Documents\pharmapulse>javac -cp "lib/*" -d bin src/*.java

C:\Users\madhv\Documents\pharmapulse>java -cp "bin;lib/*" PharmaPulseApp
Enter Admin Access Code: pp1234
===== PHARMAPULSE SYSTEM =====
----- MAIN MENU -----
1. Product Inventory
2. Orders & Billing
0. Terminate Session

Enter Your Choice: █
```

- **Inventory table listing loaded medicines:**

```

----- INVENTORY MENU -----
1. View Products
2. Add Products
3. Remove Products
4. Update Products
0. Return to Main Menu

Enter Your Choice: 1

----- PHARMAPULSE INVENTORY -----
PID      PName          Brand      PType      Stock    Cost
-----
101      Vildagliptin    Cipla      Tablet     95       120.00
102      Azimax-500     Cipla      Tablet     195      118.00
103      Cetirizine      Dr.Reddy   Tablet     80       35.00
104      Asthakind       Mankind    Syrup      75       190.00
105      Meftal-Spas     Meftal     Tablet     55       60.00
107      Montair-LC      Cipla      Tablet     100      50.00

Press 0 to exit: █

```

- **Placing an order and the background thread persisting the transaction (2 → 2).**

```

----- NEW ORDER ENTRY -----
Enter Order No.: 5
Enter Customer Name: Alice
Enter Product Name (or '0' to finish cart): Montair-LC
Enter Quantity: 5
Enter Product Name (or '0' to finish cart): 0
Enter Date (YYYY-MM-DD): 2026-09-17
>> [SUCCESS] Order persisted concurrently. Total: INR 250.0
1. AddMore
2. Exit
Choice: 2

```

- **Invoice generation output and content of the generated .txt file (2 → 4).**

```

-----GENERATE INVOICE-----
Enter Customer Name: Alice

----- PHARMAPULSE INVOICE -----
Customer: Alice
OrderNO.   Product & Quantity          Amount    Date
-----
1          Asthakind:2 | Cetirizine:2 | 450.00    2026-09-16
3          Meftal-Spas:3 | 180.00    2026-09-17
5          Montair-LC:5 | 250.00    2026-09-17

Grand Total: INR 880.00
>> [I/O STREAM] Physical receipt generated: invoice_Alice.txt

Press 0 to Exit: █

```

10. Challenges & Engineering Takeaways:

- **Thread-CLI Coordination:** Coordination between *OrderProcessorThread*, which ran in the background, and CLI main loop was possible only via *Thread.join()* to ensure that the user got the response only after completion of database write operation.
- **Early Stock Check:** Instead of creating database connections and performing subtraction in the thread itself, it was done within the preparation of cart itself.
- **Minimalistic JDBC Code:** Functional Interfaces were used (*SQLConsumer*, *SQLFunction*) to reduce boiler plate JDBC code in Service classes.

11. Future Scope:

- Refactoring order model into normalized database using a separate junction table called *order_items* along with proper foreign keys.
- Developing connection details inside system environment variable and admin role.
- Storing database connection details inside system environment variables and removing it from *db.properties* file.
- Writing Junit tests to test edge case scenarios with mocking database calls.

12.References:

- Herbert Schildt, *Java: The Complete Reference*, 11th Edition, Oracle Press, 2018.
- Oracle Java SE Documentation: Concurrency & JDBC API tutorials.
- MySQL 8.0 Reference Manual: Transaction Isolation & PreparedStatement performance